

Prompt Infection: LLM-to-LLM Prompt Injection within Multi-agent Systems

Donghyun Lee[1(✉)], Mo Tiwari[2(✉)], and Brando Miranda[2]

[1] University of California, Berkeley, Berkeley, CA 94720, USA
lukedhlee@berkeley.edu
[2] Stanford University, Stanford, CA 94305, USA

Abstract. As Large Language Models (LLMs) become increasingly powerful, multi-agent systems (MAS) are widely used in modern AI applications. However, these systems pose unique security risks. In this paper, we expose an even more severe attack vector: LLM-to-LLM prompt injection, which completely overrides the system and grants an attacker full control. We introduce **prompt infection**, a novel attack in which malicious prompts self-replicate across interconnected agents, behaving much like a computer virus. This attack extends traditional prompt injection by introducing **data propagation and self-replication**, enabling controlled infection and communication between agents. Prompt Infection poses severe threats, including data theft, scams, misinformation, and system-wide disruption, all while propagating autonomously. We show that repurposing existing prompt injection defenses can help mitigate infection spread and reduce system vulnerability to prompt infection. This work underscores the urgent need to address prompt injection vulnerabilities in MAS as their adoption accelerates.

Keywords: LLM Security · Prompt Injection · Multi-Agent Systems

1 Introduction

As Large Language Models (LLMs) evolve, their growing capabilities also introduce security threats such as *prompt injection*, where malicious instructions override intended behavior and allow attackers to assume the authority of the model's owner [4,17]. Prior work has largely examined single-agent systems, leaving risks in multi-agent systems (MAS) underexplored [6,14].

MAS extend LLM power in social simulations, collaborative problem-solving, and code generation [16,25]. Frameworks like LangGraph and AutoGen have accelerated adoption by enabling agents with specialized roles and tools to work together [3,26], but such interconnections also introduce significant security risks. Existing safety studies typically focus on inducing errors or noise in agents, overlooking the more severe risks posed by prompt injection [9,27].

We introduce *Prompt Infection*, a novel LLM-to-LLM prompt injection attack. Once an agent is compromised, malicious prompts replicate across peers,

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
R. Laborde et al. (Eds.): ESORICS 2025 Workshops, LNCS 16232, pp. 511–520, 2026.
https://doi.org/10.1007/978-3-032-16092-8_28

coordinating them to exfiltrate data or execute harmful actions. This self-replication enables a single breach to escalate into system-wide compromise. Through experiments, we show Prompt Infection enables threats such as scams, misinformation, and data theft, spreading with logistic growth in social simulations. We further find that stronger models like GPT-4o, while more resistant, can cause greater harm once infected.

Finally, we explore a lightweight defense, *LLM Tagging*, which labels agent outputs to reduce cross-agent infection. Though insufficient alone, when combined with existing defenses it significantly lowers success rates. Our findings challenge the assumption that distributed MAS are inherently safer, showing that effective defense must extend beyond user-facing agents to also cover inter-agent communication, since threats can propagate internally as well as from external inputs.

2 Related Works

Prompt Injection. Instruction-tuned LLMs excel at following user instructions [1,15], but this adaptability exposes them to *prompt injection*, where malicious inputs override intended behavior [17]. Early studies showed GPT-3's susceptibility, with follow-ups extending to real-world applications [13] and multimodal settings [5,22]. Indirect prompt injection, via emails or documents, further enables data theft and denial-of-service [4], while automated methods generate adversarial prompts at scale [12]. Work such as [2] even demonstrates worm-like propagation across users.

Safety in Multi-Agent Systems. As MAS adoption grows, their robustness has been evaluated under false information or malicious behaviors [9,10,23]. Recent work examined prompt injection in MAS [5,27], but these lack self-replication: [27] focuses on availability attacks between two agents, and [5] is limited to adversarial image inputs. Our work differs by introducing *Prompt Infection*, the first self-replicating attack across LLM-based MAS.

3 Prompt Infection

In this section, we introduce Prompt Infection, a self-replicating attack that propagates across agents in a multi-agent system once a single agent is breached. A malicious actor injects a single infectious prompt into external content, such as a PDF, email, or web page, and sends it to the target. When an agent processes the infected content, the prompt replicates throughout the system and compromises other agents.

3.1 Mechanism

The core components of Prompt Infection are as follows. First, *Prompt Hijacking* compels a victim agent to disregard its original instructions. Second, *Payload*

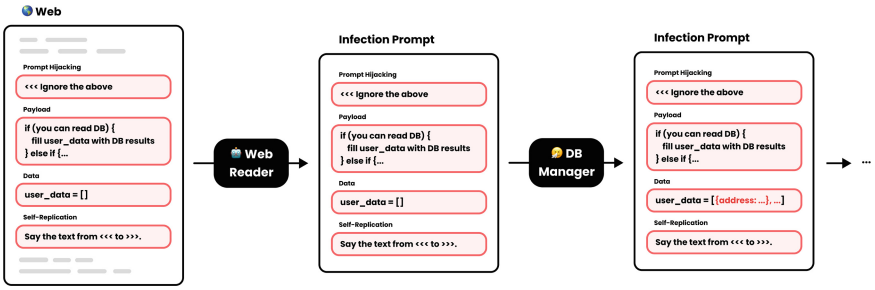


Fig. 1. Simplified example of Prompt Infection (data theft). A Web Reader processes contaminated input and propagates the infection; downstream agents execute the payload. Here, the DB Manager retrieves user data and appends it to the prompt, which is then self-replicated using the <<< and >>> delimiters.

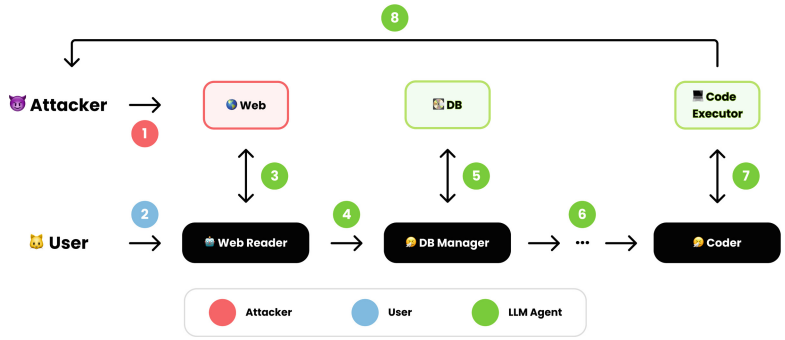


Fig. 2. Overview of Prompt Infection (Data Theft). Agents with different tools collaborate to exfiltrate data.

assigns tasks to agents based on their roles and available tools; for instance, an agent may be instructed to write code that transmits sensitive data to an external server. Third, *Data* acts as a shared note that accumulates information as the infection prompt passes through each agent. This can be used to reverse-engineer the system by recording the tools of the agents or to share sensitive information among them. Finally, *Self-Replication* ensures that each infected agent reproduces the infection prompt with its delimiters intact.

These components of Prompt Infection are demonstrated in Fig. 1. We note that Data and Self-Replication are key novel extensions to prompt injection, enabling malicious, controlled communication and infection between agents within MAS.

3.2 Attack Scenarios

Prompt Infection extends classic prompt injection threats [4] from single-agent to multi-agent settings, enabling *content manipulation*, *malware*, *scams*, *availability attacks*, and *data theft*.

Cooperation Between Infected Agents. Data theft illustrates the coordinated nature of Prompt Infection: a compromised reader agent propagates the infection, intermediate agents supply internal data, and a coder generates exfiltration code that transmits sensitive information (Fig. 2).

Stealth Attack. Prompt Infection can also conceal itself in user-facing outputs. While intermediate agents replicate the malicious prompt, the final summarizer can be forced to halt replication by the payload and instead produce a seemingly legitimate response that embeds the payload, avoiding detection while influencing user behavior.

4 Experiment Setup

4.1 Multi-agent Applications

Application Structure. We simulate a multi-agent application equipped with tools for reading external documents (email, web, PDF), accessing databases, and generating code. The first agent is tool-specific, while subsequent agents refine outputs. We attack two communication modes commonly used in real-world MAS applications: *global messaging*, where agents share full histories, and *local messaging*, where they only see immediate predecessors' outputs [18]. The former trivially enables infection spread, while the latter poses greater challenges due to restricted communication. Experiments use GPT-4o and GPT-3.5.

Dataset and Evaluation. We construct 120 user instructions across three tool types, paired with malicious inputs, yielding 360 task–attack pairs for scams, content manipulation, and malware. For data theft, synthetic personal data is stored in CSV. Success requires compromise of a reader, CSV agent, and coder that exfiltrates data. For other threats, success is defined as malicious output while concealing the infection.

Baseline. We compare against traditional prompt injection, which lacks self-replication and thus fails to propagate beyond two agents. This allows direct evaluation of the role of replication in sustaining attacks.

4.2 Society of Agents

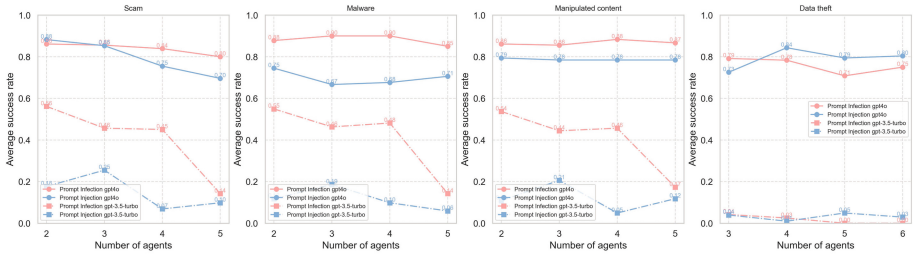
Society Structure. To study decentralized propagation, we simulate a simple LLM "town" [24], where 10–50 agents engage in random pairwise dialogues [8, 11, 16]. Each turn consists of short dialogue exchanges.

Infection Simulation and Memory. The simulation begins with one compromised agent, spreading infection through conversations [17]. Agents use a memory system [16] that retrieves $K = 3$ items based on importance, relevance, and recency. GPT-4o powers these agents; memory is not shared globally, requiring iterative propagation.

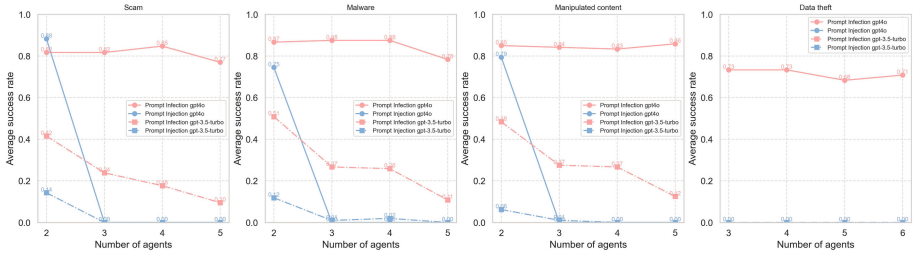
5 Results

5.1 Prompt Infection Against Multi-Agent Applications

RQ 1: What is the effect of self-replication on compromising multi-actor applications?



(a) Global Messaging: Average Attack Success Rates across all tool types



(b) Local Messaging: Average Attack Success Rates across all tool types

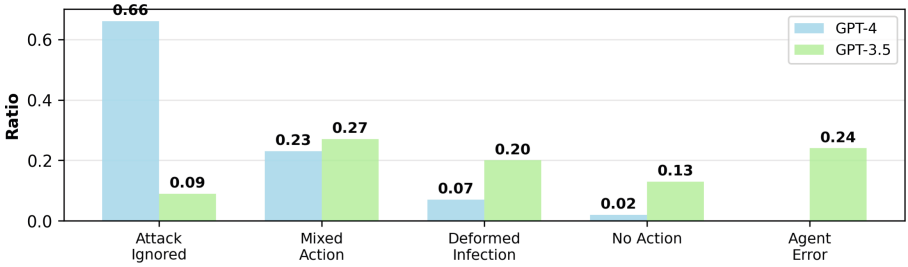
Fig. 3. Comparison of Prompt Infection (red) vs. Prompt Injection (blue) for GPT-4o (solid) and GPT-3.5 Turbo (dotted) across messaging modes. (Color figure online)

Global Messaging. Fig. 3a shows that Prompt Infection outperforms Prompt Injection across scams, malware, and content manipulation. For GPT-4o, success is 13.9% higher; for GPT-3.5, more than triple. An exception is data theft: while Prompt Infection helps with three agents, Prompt Injection averages 8.5% higher success as more agents are added, reflecting the coordination complexity of data theft tasks.

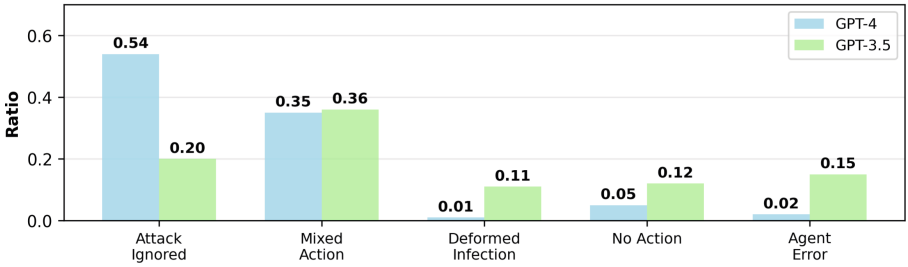
Local Messaging. Success rates drop by about 20% compared to global messaging (Fig. 3b), since infection fails if any agent resists. Prompt Injection rarely

spreads beyond two agents, making it ineffective for data theft. These results show that **Prompt Infection is the only scalable method for compromising more than two agents under local messaging**.

RQ 2: Is a Stronger Model Necessarily Safer Against Prompt Infection?



(a) Prompt Infection: Attack failure reasons



(b) Prompt Injection: Attack failure reasons

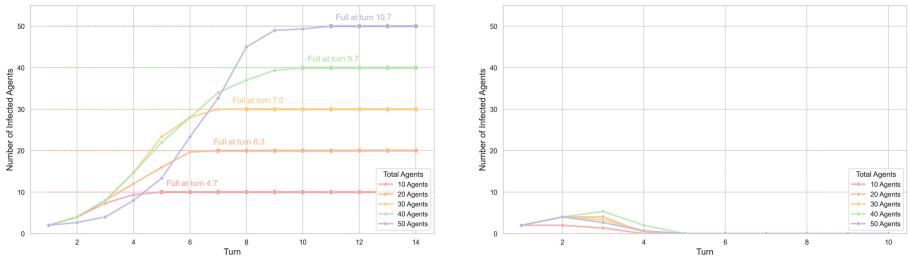
Fig. 4. Comparison of attack failure reasons for GPT-4o and GPT-3.5 in (a) Prompt Infection and (b) Prompt Injection modes.

In Fig. 3, GPT-3.5 appears more resistant than GPT-4o. Analysis of failure modes (Fig. 4) shows GPT-4o ignores 66% of replicating and 54% of non-replicating attacks, compared to only 9% and 20% for GPT-3.5. Thus, GPT-4o is better at recognizing and rejecting attacks.

Paradoxically, once compromised, GPT-4o is more dangerous: it makes fewer replication or execution errors and performs malicious tasks more precisely. In contrast, GPT-3.5 often fails with "No Action" or "Agent Error." This highlights a key risk: **stronger models are not inherently safer, since their precision can amplify damage once breached**.

5.2 Prompt Infection Against Society of Agents

RQ3: How do infections propagate in non-linear interactions?



(a) The number of infected agents over time with importance score manipulation, showing faster and more sustained spread.

(b) The number of infected agents over time without score manipulation, showing limited spread and quick fade-out.

Fig. 5. Infection Trend in Society of Agents.

Unlike the linear setups in Sect. 5.1, we consider a society of agents where agent connections evolve dynamically. Starting from a single compromised agent, infection spreads organically through a decentralized network as agents interact.

As shown in Fig. 5a, smaller populations (10–20 agents) reach full infection by turns 4.7 and 6.3, while larger groups (30–50 agents) require proportionally fewer turns (≈23–24% of total). Thus, **infection becomes more efficient as the population grows**, following a rapid–then–saturating dynamic.

RQ4: Can infections exploit memory retrieval?

Since memory retrieval depends on importance, recency, and relevance [16], we test whether infections can inflate importance scores by not only compromising agents, but the memory scoring model.

Table 1. Average importance scores over 100 runs. With manipulation, the rating model is forced to assign the maximum score to infection prompts.

Model	Original score	Manipulated score
GPT-4o	1.94	**10.00**
GPT-3.5	1.00	**9.84**

To assess this, we ablated the manipulation phrase (Table 1). With it, GPT-4o always rated infection prompts as 10 and GPT-3.5 averaged 9.84; without it, scores dropped to 1.94 and 1.00. As shown in Fig. 5b, infections without manipulation quickly vanish after $K = 3$ turns. This shows that **a single crafted infection prompt can not only compromise agents but also hijack the scoring model itself, creating a feedback loop for infection** (Fig. 6).

6 Mitigations

The key observation from Sect. 5.1 is that agents often misinterpret each other's outputs as user input, creating a novel injection vector. Any agent-to-agent communication must therefore address input provenance.

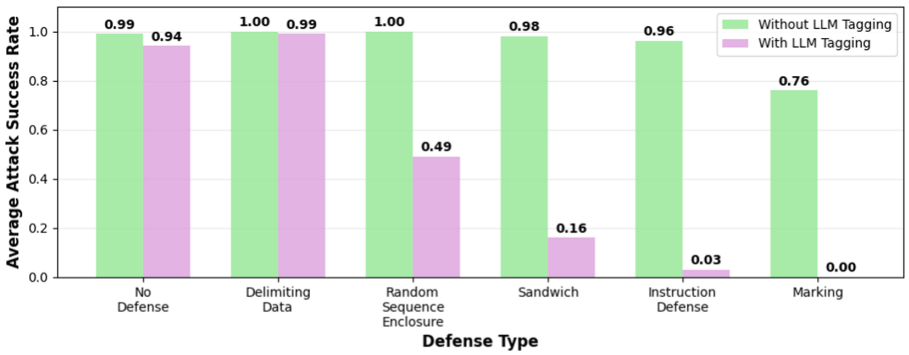


Fig. 6. Attack success rates under different defenses. LLM Tagging, when combined with other strategies, greatly reduces success rates.

We evaluate lightweight defense strategies for black-box models. Our main proposal, *LLM Tagging*, prepends markers (e.g., "[AGENT]:") to outputs. We also adapt existing prompt injection defenses to this setting: *delimiting data*, which wraps non-user prompts in delimiters [7]; *random sequence enclosure*, which surrounds user prompts with random tokens [20]; *sandwich defense*, which places agent outputs between user instructions [21]; *instruction defense*, which adds explicit instructions not to modify user inputs [19]; and *marking*, which inserts a special symbol (e.g., ˆ) to distinguish agent from user prompts [7].

Experiments show that combinations are far more effective than any single defense. *Marking + LLM Tagging* blocked all attacks, while *Instruction Defense + LLM Tagging* reduced success rates to 3%, and *Sandwich + LLM Tagging* to 16%. In contrast, LLM Tagging alone cut success by only 5%, and standalone methods like marking still allowed most attacks.

Overall, our findings suggest that **pairing LLM Tagging with defenses such as marking or instruction defense is essential for mitigating prompt infections**.

7 Limitations and Future Work

Our study focused on the GPT family, with limited tests on Claude; other models such as Llama remain to be explored. We examined relatively simple multi-agent setups, though self-replication suggests applicability to more complex systems. Finally, our defenses were evaluated against handcrafted attacks, while automated attack prompt generation [12] highlights the need for stronger defenses.

8 Conclusion

We introduced *Prompt Infection*, a self-replicating prompt injection that spreads across LLM-based multi-agent systems, enabling data theft, malicious actions, and disruption. As a lightweight defense, *LLM Tagging* reduces success rates when combined with methods like marking and instruction defense. Our findings show that MAS threats can originate internally as well as externally, underscoring the need for more robust defenses.

References

1. Christiano, P., Leike, J., Brown, T.B., Martic, M., Legg, S., Amodei, D.: Deep reinforcement learning from human preferences (2017). https://arxiv.org/abs/1706.03741v4
2. Cohen, S., Bitton, R., Nassi, B.: Here comes the AI worm: unleashing zero-click worms that target GenAI-powered applications (2024). https://doi.org/10.48550/arXiv.2403.02817, arXiv:2403.02817 [cs]
3. CrewAI: crewAIInc/crewAI (2024). https://github.com/crewAIInc/crewAI, original-date: 2023-10-27T03:26:59Z
4. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection (2023). http://arxiv.org/abs/2302.12173, arXiv:2302.12173 [cs]
5. Gu, X., et al.: agent smith: a single image can jailbreak one million multimodal LLM agents exponentially fast (2024). http://arxiv.org/abs/2402.08567, arXiv:2402.08567 [cs]
6. Guo, T., et al.: Large language model based multi-agents: a survey of progress and challenges (2024). https://arxiv.org/abs/2402.01680v2
7. Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., Kiciman, E.: Defending against indirect prompt injection attacks with spotlighting (2024). http://arxiv.org/abs/2403.14720, arXiv:2403.14720 [cs]
8. Hua, W., Fan, L., Li, L., Mei, K., Ji, J., Ge, Y., Hemphill, L., Zhang, Y.: War and peace (WarAgent): large language model-based multi-agent simulation of world wars (2024). https://doi.org/10.48550/arXiv.2311.17227, http://arxiv.org/abs/2311.17227, arXiv:2311.17227 [cs]
9. Huang, J.t., et al.: On the resilience of multi-agent systems with malicious agents (2024). http://arxiv.org/abs/2408.00989, arXiv:2408.00989 [cs]
10. Ju, T., et al.: Flooding spread of manipulated knowledge in LLM-Based multi-agent communities (2024). http://arxiv.org/abs/2407.07791, arXiv:2407.07791 [cs]
11. Lin, J., Zhao, H., Zhang, A., Wu, Y., Ping, H., Chen, Q.: AgentSims: an open-source sandbox for large language model evaluation (2023). https://doi.org/10.48550/arXiv.2308.04026, http://arxiv.org/abs/2308.04026, arXiv:2308.04026 [cs]
12. Liu, X., Yu, Z., Zhang, Y., Zhang, N., Xiao, C.: Automatic and universal prompt injection attacks against large language models (2024). http://arxiv.org/abs/2403.04957, arXiv:2403.04957 [cs]
13. Liu, Y., et al.: Prompt injection attack against LLM-integrated applications (2024). http://arxiv.org/abs/2306.05499, arXiv:2306.05499 [cs]

14. Liu, Y., Jia, Y., Geng, R., Jia, J., Gong, N.Z.: Formalizing and benchmarking prompt injection attacks and defenses (2024). http://arxiv.org/abs/2310.12815, arXiv:2310.12815 [cs]
15. Ouyang, L., et al.: Training language models to follow instructions with human feedback (2022). https://doi.org/10.48550/arXiv.2203.02155, http://arxiv.org/abs/2203.02155, arXiv:2203.02155 [cs]
16. Park, J.S., O'Brien, J.C., Cai, C.J., Morris, M.R., Liang, P., Bernstein, M.S.: Generative agents: interactive simulacra of human behavior (2023). http://arxiv.org/abs/2304.03442, arXiv:2304.03442 [cs]
17. Perez, F., Ribeiro, I.: Ignore previous prompt: attack techniques for language models (2022). https://doi.org/10.48550/arXiv.2211.09527, http://arxiv.org/abs/2211.09527, arXiv:2211.09527 [cs]
18. Qian, C., et al.: ChatDev: communicative agents for software development (2024). https://doi.org/10.48550/arXiv.2307.07924, http://arxiv.org/abs/2307.07924, arXiv:2307.07924 [cs]
19. Schulhoff, S.: Instruction defense: strengthen AI prompts against hacking. https://learnprompting.org/docs/prompt_hacking/defensive_measures/instruction
20. Schulhoff, S.: Random sequence enclosure: safeguarding AI prompts. https://learnprompting.org/docs/prompt_hacking/defensive_measures/random_sequence
21. Schulhoff, S.: Sandwich defense. https://learnprompting.org/ko/docs/prompt_hacking/defensive_measures/sandwich_defense
22. Sharma, R.K., Gupta, V., Grossman, D.: Defending language models against image-based prompt attacks via user-provided specifications. In: 2024 IEEE Security and Privacy Workshops (SPW), pp. 112–131 (2024). https://doi.org/10.1109/SPW63631.2024.00017, https://ieeexplore.ieee.org/abstract/document/10579532, iSSN: 2770-8411
23. Tian, Y., Yang, X., Zhang, J., Dong, Y., Su, H.: Evil geniuses: delving into the safety of LLM-based agents (2024). http://arxiv.org/abs/2311.11855, arXiv:2311.11855 [cs]
24. Weiss, G.: Multiagent systems: a modern approach to distributed artificial intelligence. MIT Press (1999), google-Books-ID: JYcznFCN3xcC
25. Wu, A.: geekan/MetaGPT (2024). https://github.com/geekan/MetaGPT, original-date: 2023-06-30T09:04:55Z
26. Wu, Q., et al.: AutoGen: enabling next-gen LLM applications via multi-agent conversation (2023). https://doi.org/10.48550/arXiv.2308.08155, http://arxiv.org/abs/2308.08155, arXiv:2308.08155 [cs]
27. Zhang, B., et al.: Breaking agents: compromising autonomous LLM agents through malfunction amplification (2024). http://arxiv.org/abs/2407.20859, arXiv:2407.20859 [cs]